

LAB 4: Stochastic gradient descent - Documentation

The aim of this lab is to implement the design of Seminar 4, which extends our previous linear-regression framework with two new ideas: **standardized datasets** using an abstract Dataset hierarchy, and **stochastic gradient descent (SGD)** as an alternative optimization algorithm.

We reused the classes Vector, Record, and Model from earlier labs, and added or adapted several new classes required by the new design. Finally, we created a TestLearner class to verify that all components interact correctly for both raw and standardized datasets.

Overall, our program is able to transform datasets (raw or standardized), compute either full-batch or stochastic gradients, learn a linear-regression model, and produce predictions on new inputs.

Classes, attributes, and methods

Classes we reused or modified:

1. **abstract Dataset**: This class no longer computes means or standard deviations; these are entirely delegated to RawDataset (5)
 - **Attributes:**
 - *int dim*
 - *List<Record> data*
 - **Methods:**
 - *Dataset(int d)*: constructor
 - *getDim()* → *int*, *getData()* → *List <Record>*: getters
 - *round5(double val)* → *double*: helper method to round the values (5 decimals)
 - *addRecord(Record r)*: adds a record to the dataset
 - *abstract transform(Record r)* → *Record*
 - *abstract output(double y)* → *double*: reverse transformation
 - *toString()* → *String*: returns a readable string of dataset
2. **StandardizedDataset** extends Dataset: Represents the dataset after standardization
 - **Attributes:**
 - *Vector mi*: mean of the input
 - *Vector si*: standard deviation of the input
 - *double mo*: mean of the output
 - *double so*: standard deviation of the output
 - **Methods:**
 - *StandardizedDataset(int d, Vector mi, Vector si, double mo, double so)*: constructor
 - *getMeanIn()* → *Vector*, *getStdIn()* → *Vector*, *getMeanOut()* → *double*, *getStdOut()* → *double*: getters
 - *transform(Record r)* → *Record*: standardizes using $x' = (x - m_i) / s_i$ and $y' = (y - m_o) / s_o$
 - *output(double d)* → *double*: Applies the reverse transformation ($y = y' \cdot s_o + m_o$)
3. **abstract Algorithm**: Represents any supervised learning algorithm
 - **Attributes:**
 - *double learningRate*
 - **Methods:**
 - *Algorithm(double lr)*: constructor
 - *getLearningRate()*: getter
 - *abstract solve(Dataset ds)* → *Model*

4. **SupervisedLearner**: Modified the method `predict()` as follows:
- Before making a prediction, the input vector must be wrapped into a temporary `Record` so that `ds.transform()` can be applied.
 - After getting the model's prediction (which may be in standardized space), we apply `ds.output()` to return a prediction in the original output space.
 - This ensures the learner works both for raw and standardized datasets.

New classes we implemented:

5. **RawDataset** extends Dataset: Represents an untransformed dataset and computes all statistics needed for standardization
- **Methods**:
 - `RawDataset(int d)`: constructor
 - `meanInput()` → `Vector`: computes the mean of the input
 - `stdInput()` → `Vector`: computes the standard deviation of the input
 - `meanOutput()` → `double`: computes the mean of the output
 - `stdOutput()` → `double`: computes the standard deviation of the output
 - `standardize()` → `StandardizedDataset`: computes all statistics and transforms the raw dataset into a standardized one
 - `transform(Record r)` → `Record`: returns unchanged `r`
 - `output(double d)` → `double`: returns unchanged `d`
6. **GradientDescent** extends Algorithm: implements full-batch gradient descent (GD)
- **Attributes**:
 - `double stoppingCriterion`
 - **Methods**:
 - `GradientDescent(double lr, double sc)`: constructor
 - `getStoppingCriterion()` → `double`: getter
 - `gradient(Dataset ds, Model m)` → `Vector`: computes the gradient vector
 - `solve(Dataset ds)` → `Model`: Performs gradient descent by repeatedly computing the gradient and updating the model until convergence, and returns the learned `Model`
7. **StochasticGradientDescent** extends Algorithm: implements a smaller-batch GD (SGD)
- **Attributes**:
 - `int batchSize`
 - `int iterations`
 - `Random random`
 - **Methods**:
 - `StochasticGradientDescent(double lr, int bs, int i)`: constructor
 - `stochasticGradient(Dataset ds, Model m)` → `Vector`: Samples a minibatch of records using and computes the average gradient
 - `solve(Dataset ds)` → `Model`: Runs SGD for a fixed number of iterations, updating the `Model` after each minibatch. Returns the learned `Model`
8. **TestLearner**: We tested all components printing results and an error counter. Tests include:
- Checking that raw datasets and standardized datasets transform correctly
 - Verifying stochastic gradients are not zero vectors
 - Testing that SGD and full gradient descent produce similar final models
 - Ensuring the reverse transformation (`output()`) works
 - Verifying predictions on simple linear datasets

Before coding, we discussed several possible designs. One option was to keep all mean and standard-deviation computations inside *Dataset*, as in Lab 2. This would have simplified the implementation of *RawDataset*, but it would contradict *Dataset* abstraction.

We preferred the required design because it cleanly separates responsibilities:

- *Dataset* groups behavior
- *RawDataset* computes statistics and can generate a standardized dataset
- *StandardizedDataset* performs the actual transformations

Another design alternative was to manually construct minibatches by creating intermediate lists of sampled *Records*. Instead, we followed the handout's suggestion and used *Random.ints().distinct().limit()*. This avoids unnecessary data structures and makes the code more efficient and compact.

We also discussed whether to apply *output()* inside the *Model* class instead of *SupervisedLearner*. However, this would mix learning logic with dataset logic, making *Model* depend on *Dataset*. We preferred the cleaner design where *SupervisedLearner* handles all interactions between components.

This lab applied several object-oriented programming concepts:

- **Inheritance:** *RawDataset* and *StandardizedDataset* both extend *Dataset* and override its abstract methods.
- **Abstraction:** *Dataset* hides transformation logic behind abstract methods *transform()* and *output()*.
- **Encapsulation:** All attributes remain private, accessed only via getters.
- **Polymorphism:** Both *GradientDescent* and *StochasticGradientDescent* can be used interchangeably through the *Algorithm* interface.

In practice, our implementation worked as expected. *TestLearner* confirmed that:

- *StandardizedDataset* correctly transforms input after calling *transform()*
- The new *predict()* method correctly transforms and reverses transformation
- *GradientDescent* converges to a stable model using the stopping criterion
- *StochasticGradientDescent*, properly tuned, produces a model similar to full *GradientDescent*
- Both *RawDataset* and *StandardizedDataset* run without errors, and predictions stay consistent across both approaches
- All classes integrated smoothly with those from previous labs

We faced some difficulties during the implementation. A particularly important part of the lab was tuning the parameters (*learningRate*, *batchSize* and *iterations*) for *StochasticGradientDescent*. After experimenting with several learning rates, we ultimately chose the largest *learningRate* that still produced a stable result, then adjusted *iterations* to the minimum value that also gave a stable output. This keeps the algorithm efficient, avoids unnecessary updates and makes SGD behave predictably while still being faster per iteration than full gradient descent.

After fixing these issues and selecting good parameters, the final solution passed all our tests with 0 errors. Overall, this lab helped us better understand abstraction, inheritance, and how stochastic optimization fits into an object-oriented framework, as well as the practical side of tuning learning algorithms.